Object-Oriented Programming

4. Encapsulation, ADTs and Pattern Matching

Filip Křikava

https://courses.fit.cvut.cz/BI-OOP/

Any questions?

Encapsulation

Public fields let any caller overbook

```
final class SeatPool(var capacity: Int, var reserved: Int)

val seats = SeatPool(100, 99)

seats.reserved += 2

seats.capacity - seats.reserved   // -1 available seats
```

Who is responsible for keeping 0 <= reserved <= capacity?

Private fields are only the first step

```
final class SeatPool(val capacity: Int):
  require(capacity >= 0)
  private var _reserved = 0
  def reserved: Int = _reserved
  def reserved_=(n: Int): Unit =
    require(0 <= n && n <= capacity)
    _reserved = n

val seats = SeatPool(100)
// Repeated in checkout, kiosk, group booking, ...
if seats.reserved + 2 <= seats.capacity then
  seats.reserved = seats.reserved + 2
```

The setter checks a range. Every client still implements booking.

Expose a booking operation

```
final class SeatPool(val capacity: Int):
  require(capacity >= 0)
  private var reserved = 0

  def available: Int = capacity - reserved

  def tryReserve(n: Int): Boolean =
    require(n > 0)
    if n > available then false
    else
      reserved += n
      true

val seats = SeatPool(100)
seats.tryReserve(2)   // true; available is now 98
```

Single-threaded model: construction and tryReserve preserve the invariant.

Change storage without changing callers

```
final class SeatPool(val capacity: Int):
  require(capacity >= 0)
  private var free = capacity

  def available: Int = free

  def tryReserve(n: Int): Boolean =
    require(n > 0)
    if n > free then false
    else
      free -= n
      true

// Same API and results; no stored reservation count.
```

The public contract survives a change from reserved to free.

A boundary makes bugs local

```
val seats = SeatPool(3)

assert(seats.tryReserve(2))
assert(!seats.tryReserve(2))
assert(seats.available == 1)
```

If this fails, inspect the SeatPool implementation.

Checkout, kiosk and group booking cannot write the internal state.

A small API turns a whole-application problem into a small object-level test.

Controlling member accessibility

Turn the intended audience of a member into compiler rules.

Modifier	Who can access it?	Can a subclass override it?
public (default)	Any client	Yes, unless final
protected	Class, companion, subclasses	Yes, unless final
private	Class and companion	No

Choose the smallest audience. Accessibility and overriding are separate decisions.

Access modifiers - public

```
class Super:
  def f() = println("f")

class Sub extends Super:
  f()

class Other:
  new Super().f()
```

- Scala default (differs from Java)
- Public members are accessible from everywhere
  - they can be called
  - they can be overridden (unless defined `final`)

Access modifiers: protected

```
class Super:
  protected def f(): Unit = println("f")

class Sub extends Super:
  f()   // allowed in a subclass

class Other:
  Super().f()   // compile error: protected
  Sub().f()     // compile error: protected
```

Accessible in the defining class, its companion, and subclasses.
Unlike Java, sharing a package does not grant access.
protected final allows calls from subclasses but prevents overriding.

Access modifiers and subtyping

```
class Super:
  protected def f(): String = "parent"

class Sub extends Super:
  override def f(): String = "child"   // now public
```

```
Super().f()   // compile error: protected
Sub().f()     // "child"
```

The override is public by default. The parent declaration is still protected.

Optional

Access modifiers - private

```
class Super:
  private def f() = { println("f") }
  f() // ok

class Sub extends Super:
  f() // symbol f is inaccessible from this point

class Other:
  new Super().f() // symbol f is inaccessible from this point
  new Sub().f()   // symbol f is inaccessible from this point
```

- Private members are accessible only from
  - inside the class or
  - companion object

Access modifiers and subtyping

```
class Super:
  private def f(): String = "parent"

class Sub extends Super:
  override def f(): String = "child"   // now public
```

```
Super().f()   // compile error: private
Sub().f()     // compile error but at the override definition:
              //    it cannot override a private method!
```

Private methods cannot be overridden

Optional

A private reference can still leak state

```
import java.util.Date

final class TimeWindow(private val start: Date,
                       private val end: Date):
  require(start.before(end))
  def startsAt: Date = start

val start = new Date(0)
val window = TimeWindow(start, new Date(1000))

start.setTime(2000)              // input alias
window.startsAt.setTime(3000)    // output alias
```

Both mutations bypass the object. The start is now after the end.

Copy mutable values at both boundaries

```
import java.util.Date

final class TimeWindow(start: Date, end: Date):
  private val from = new Date(start.getTime)
  private val until = new Date(end.getTime)
  require(from.before(until))

  def startsAt: Date = new Date(from.getTime)
  def endsAt: Date = new Date(until.getTime)

val input = new Date(0)
val window = TimeWindow(input, new Date(1000))
input.setTime(2000)              // changes input only
window.startsAt.setTime(3000)    // changes a copy only
```

Copy on entry and on exit. Validate the copies that the object owns.

Immutable values make sharing safe

```
import java.time.Instant

final class TimeWindow(val start: Instant, val end: Instant):
  require(start.isBefore(end))

val window = TimeWindow(Instant.EPOCH,
                        Instant.EPOCH.plusSeconds(60))

val later = window.start.plusSeconds(3600)

window.start == Instant.EPOCH   // true
```

Instant creates new values. Sharing these references cannot change the window.

A collection getter can bypass validation

```
import scala.collection.mutable.ArrayBuffer

final class Role(val name: String, var active: Boolean)

final class Team:
  private val members = ArrayBuffer.empty[Role]
  def add(role: Role): Unit =
    require(role.active)
    members += role
  def roles: ArrayBuffer[Role] = members

val team = Team()
team.roles += Role("admin", false)
```

Policy: every member is active. The getter lets callers skip add.

An immutable container is a shallow fix

```
// Replace the getter in Team:
def roles: Vector[Role] = members.toVector

val admin = Role("admin", true)
val team = Team()
team.add(admin)

admin.active = false              // input alias
team.roles.head.active = false    // output alias

team.roles.head.active            // false
```

The Vector cannot change. Its Role objects can still change.

Share immutable snapshots

```
final class Role(val name: String, val active: Boolean)

final class Team:
  private var members = Vector.empty[Role]
  def add(role: Role): Unit =
    require(role.active)
    members = members :+ role
  def roles: Vector[Role] = members

val team = Team()
team.add(Role("admin", true))
val disabled = team.roles.head // can access, but cannot change the state of the role
```

The container and its elements are immutable. All membership changes go through Team.

Control construction with a companion

```
final class Percentage private (val value: Int)

object Percentage:
  def apply(n: Int): Option[Percentage] =
    if 0 <= n && n <= 100 then
      Some(new Percentage(n))
    else None

Percentage(25)        // Some(...)
Percentage(150)       // None
new Percentage(150)   // compile error outside the companion
```

A private constructor makes the validated factory the entry point.

One boundary check simplifies every caller

```
def discount(price: BigDecimal, p: Percentage): BigDecimal =
  price * (100 - p.value) / 100

Percentage(25).map(p => discount(BigDecimal(200), p))
// Some(150)

Percentage(150).map(p => discount(BigDecimal(200), p))
// None: discount is never called
```

discount can rely on 0 <= p.value <= 100. It does not repeat validation.

Which API creates fewer dependencies?

```
// API A
def reserved: Int
def reserved_=(n: Int): Unit
def records: ArrayBuffer[Booking]
```

```
// API B
def available: Int
def tryReserve(n: Int): Boolean
def bookings: Iterator[BookingSummary]
```

Who can invalidate the state?
Which callers must change when storage changes?

Cohesion and coupling in the booking example

High cohesion

SeatPool keeps booking state and booking policy together.

One place to change the rule.

Low coupling

Checkout depends on tryReserve and available.

It does not know how seats are stored.

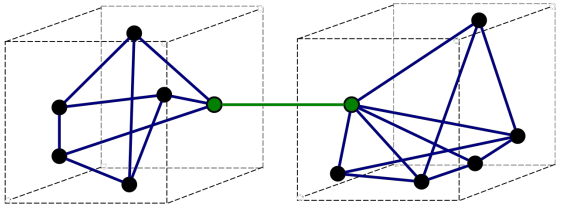
Keep each rule behind one boundary; expose the smallest useful contract.

Cohesion and coupling

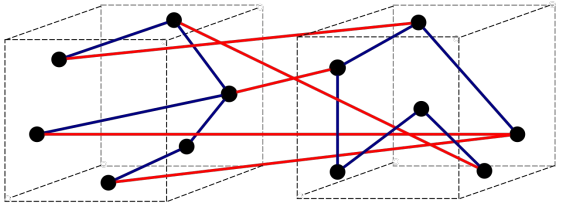
- Cohesion
  - how much elements inside a module belong together?
  - small methods, related activities
- Coupling
  - what is the interdependence between modules?

- Aim at high cohesion and low coupling

- Why?
  - A change in one module usually forces a ripple effect of changes in other modules.
  - Increased effort to assemble dependent modules
  - Harder to reuse and/or test because dependent modules must be included.



a) Good (loose coupling, high cohesion)



b) Bad (high coupling, low cohesion)

Encapsulation: protect a useful contract

Preserve the rule
Expose domain operations that keep the invariant true.

Keep callers independent
Expose behavior instead of storage details.

Own mutable state
Copy mutable inputs and outputs, or share immutable values.

Use visibility deliberately
private is a mechanism; the API is the design.

A good boundary makes invalid changes hard and implementation changes local.

Case classes and data modelling

Payments are values passed between components

```
final class Payment(val amount: BigDecimal, val currency: String)

val a = Payment(BigDecimal(120), "CZK")
val b = Payment(BigDecimal(120), "CZK")

a.amount        // 120
a.currency      // "CZK"
a == b          // true
```

For a payment value, equality should follow the fields. A plain class does not generate that API.

A case class gives the payment value semantics

```
final case class Payment(amount: BigDecimal, currency: String)

val a = Payment(BigDecimal(120), "CZK")
val b = Payment(BigDecimal(120), "CZK")
a == b          // true: the fields are equal

val corrected = a.copy(amount = BigDecimal(150))
a.amount            // 120
corrected.amount    // 150
```

copy creates a new value. Constructor fields are val by default; use immutable field types.

Case class

The val keyword is implicit in case classes

```
final case class Payment(amount: BigDecimal, currency: String)
```

```
final class Payment(val amount: BigDecimal, val currency:
Boolean):
  def toString: String = ...
  def equals(x: Any): Boolean = ...
  def hashCode(x: Any): Int = ...

  def copy(
    amount: BigDecimal = this.amount,
    currency: String = this.currency
  ): Payment = ...

object Role:
  def apply(amount: BigDecimal, currency: String): Role = ...
  def unapply(c: Role): Option[(BigDecimal, String)] = ...
```

- Model plain data aggregates with less ceremony
- Scala's take on product part of Algebraic Data Types
- Immutable (strongly recommended)
- Should not be extended (make them final)
- Next to a new type definition, the following gets generated
  - `toString()`
  - `equals()/hashCode()`
  - `copy()`
  - `product*()`
  - a factory method
  - extractor for pattern matching

Four questions that determine the model

Order situations

Draft: delivery can still change.
Paid: a payment reference exists.
Shipped: payment and tracking exist.
Cancelled: cancelled before payment.

Ask before choosing fields

Which situations are valid?
What facts belong to each one?
Which changes are allowed?
What identifies the same order?

Write representative valid and invalid examples before writing the type definitions.

Decision 1: independent facts or alternatives?

```scala
// Either preference may be enabled.
final case class Notifications(
  email: Boolean,
  sms: Boolean)

Notifications(true, true)
```

```scala
// Exactly one selected channel.
enum Channel:
  case Email, Sms

val channel = Channel.Email
```

Use a product for independent choices; use a sum when exactly one alternative is selected.

Decision 2: which facts belong to each case?

```
final case class Delivery(
  pickup: Boolean,
  address: Option[Address],
  express: Boolean)

Delivery(true, None, true)
// express pickup?

Delivery(false, None, false)
// courier without an address?
```

```
enum Delivery:
  case Pickup
  case Courier(
    address: Address,
    express: Boolean)

// Each variant has only
// the fields that it needs.
```

The variants rule out incompatible fields. Address values can still need validation.

Name identity and references by their domain meaning

```
final case class OrderId(value: String)
final case class PaymentId(value: String)
final case class TrackingCode(value: String)

final case class OrderInfo(id: OrderId, delivery: Delivery)

val payment: PaymentId = OrderId("o1")
// Compile error: different meanings have different types.
```

Products group facts. Wrappers separate meanings; field policies still need validation.

Use optional fields with care

```
final case class Order(
  id: String,
  paymentId: Option[String],
  tracking: Option[String],
  cancelled: Boolean)

Order("o1", None, Some("trk7"), false)
// Tracking exists, but no payment reference.

Order("o1", Some("pay8"), Some("trk7"), true)
// Both shipped and cancelled.
```

Both values type-check. The product permits combinations that the domain rules forbid.

Give each order state exactly its required facts

```
enum Order:
  case Draft(info: OrderInfo)
  case Paid(info: OrderInfo, payment: PaymentId)
  case Shipped(info: OrderInfo, payment: PaymentId, tracking: TrackingCode)
  case Cancelled(info: OrderInfo)
```

The sum chooses the state; each product supplies the facts required in that state.

Decision 3: which transitions may the API allow?

```
def pay(draft: Order.Draft, id: PaymentId): Order.Paid =
  Order.Paid(draft.info, id)

def dispatch(paid: Order.Paid, code: TrackingCode): Order.Shipped =
  Order.Shipped(paid.info, paid.payment, code)

def cancel(draft: Order.Draft): Order.Cancelled =
  Order.Cancelled(draft.info)

val info = OrderInfo(OrderId("o1"), Delivery.Pickup)
val draft: Order.Draft = Order.Draft(info)
dispatch(draft, TrackingCode("CZ123"))   // compile error
```

These signatures restrict inputs. Protect the application boundary when identity and history matter.

Decision 4: value equality or stable identity?

```
// A value has equality by its components.
val p = Payment(BigDecimal(120), "CZK")
val corrected =
  p.copy(amount = BigDecimal(150))

p == corrected   // false
```

```
final case class OrderSummary(
  id: OrderId, total: BigDecimal)

val old = OrderSummary(
  OrderId("o1"), BigDecimal(120))
val now =
  old.copy(total = BigDecimal(150))

old == now          // false
old.id == now.id    // true
```

A changed snapshot may represent the same entity. Model its identity explicitly.

Name each meaning of absence explicitly

```
// Does None mean loading, missing, or a failure?
val result: Option[OrderInfo] = None

enum OrderLookup:
  case Loading
  case Missing
  case Failed(reason: String)
  case Found(info: OrderInfo)
```

Use Option when absent/present is the complete distinction. Name additional domain outcomes explicitly.

Choose emptiness, ordering and duplicates deliberately

Choose the allowed number of values

`Option[A]`: zero or one
`Either[A, B]`: exactly one, either A or B
`Vector[A]`: ordered, zero or more
`Set[A]`: distinct elements
`Map[K, V]`: at most one value per key

```scala
// Require one or more elements.
final case class NonEmpty[A](
  head: A,
  tail: Vector[A])

val xs = NonEmpty(1, Vector(2, 3))

// An empty value has no constructor.
```

Ordering, duplicates and emptiness are domain decisions. Encode them deliberately.

Validate field policies at the boundary

```
final class Address private (val value: String)

object Address:
  def from(raw: String): Either[String, Address] =
    val text = raw.trim
    if text.nonEmpty then Right(new Address(text))
    else Left("Address is required")

Address.from("   ")   // Left("Address is required")
```

The ADT requires an Address. This factory enforces a minimal field policy; richer address rules can follow.

Use values when callers need to inspect the data

```scala
final case class Payment(
  amount: BigDecimal,
  currency: String)

def receipt(p: Payment): String =
  s"${p.amount} ${p.currency}"

val p = Payment(BigDecimal(120), "CZK")
receipt(p)   // "120 CZK"
```

The data shape is the contract.

Useful for messages, results, configuration and snapshots.

Consumers can format, group and transform these values independently.

Choose case classes and ADTs when exposing the cases and fields is useful.

Brian Goetz (2022): Data Oriented Programming in Java

Use an object when operations are the contract

```
trait PaymentRepository:
  def find(id: PaymentId): Option[Payment]
  def save(id: PaymentId, p: Payment): Unit

// Implementation options:
// database, network service,
// or in-memory storage.
```

Hide representation and resources.

Useful for repositories, sessions, connections and lifecycle rules.

The object may be immutable.
It can consume and return data values.

Combine them: a repository object can store and return immutable Payment values.

Brian Goetz (2022): Data Oriented Programming in Java

Pattern Matching

Pattern matching on values

```
val x = 3

val result = x match
  case 1 => "one"
  case 2 => "two"
  case 3 => "three"
  case _ => "something else"

println(result) // "three"
```

Pattern matching on values

```
def describe(x: Any): String =
  x match
    case 5 => "five"
    case true => "truth"
    case "hello" => "hi!"
    case Nil => "the empty list"
    case _ => "something else"

describe(5)   // "five"
```

Pattern matching on types

```
def describe(value: Any): String =
  value match
    case n: Int     =>  s"Int: $n"
    case s: String  =>  s"String: $s"
    case b: Boolean =>  s"Boolean: $b"
    case other      =>  s"Other: $other"

describe(10)      // Int: 10
describe("hello") // String: hello
```

Pattern matching on ADT

```
def onOption(x: Option[Int]) =
  x match
    case Some(x) => s"has value $x"
    case None    => "no value"

onOption(Some(1)) // has value 1
```

A guard can leave an ADT case uncovered

```
def onOption(x: Option[Int]): String = x match
  case Some(n) if n > 10 => s"has value $n > 10"
  case None => "no value"

onOption(Some(11))   // "has value 11 > 10"
onOption(Some(5))    // MatchError
```

```
Warning:
  Line 2: match may not be exhaustive.
          It would fail on pattern case: Some(_)
```

Cover the cases that fail the guard

```
def onOption(x: Option[Int]): String = x match
  case Some(n) if n > 10 => s"has value $n > 10"
  case Some(_) => "has some other value"
  case None => "no value"

onOption(Some(11))   // "has value 11 > 10"
onOption(Some(5))    // "has some other value"
onOption(None)       // "no value"
```

Prefer explicit ADT cases: a catch-all can hide a newly added variant.

Stable identifiers and variable patterns

```
def onOption(x: Option[Int]) =
  val A = 10
  x match
    case Some(A) => "has value 10"
    case Some(a) => s"has some other value $a"
    case None    => "no value"

onOption(Some(10)) // has value 10
onOption(Some(11)) // has some other value 11
```

Pattern matching and subtyping

```
class A
class B extends A
class C extends B

val a: A = C()

a match                 // 1
  case x: A => 1
  case x: B => 2
  case x: C => 3

a match                 // 3
  case x: C => 3
  case x: B => 2
  case x: A => 1
```

```
Warning: Unreachable case
Warning: Unreachable case
```


Optional

Pattern matching on lists

```
def sum(xs: List[Int], acc: Int = 0): Int = xs match
  case Nil => acc
  case head :: tail => sum(tail, acc + head)

sum(List(1, 2, 3))         // 6
sum(1 :: 2 :: 3 :: Nil)    // 6
```

In a pattern, `head :: tail` extracts a nonempty list.
In an expression, `::` is a right-associative method that prepends.
Nil is the empty-list case.

Combine sequence and constructor patterns

```
val xs = List(Some(1), Some(2), None)

xs match
  case Some(1) :: Some(n) :: _ :: Nil => s"Three entries; second is $n"
  case Some(1) :: _ :: _ :: _ => "At least three entries, starting with 1"
  case Nil => "Empty list"
  case _ => "Another list"

xs match
  case Seq(Some(1), Some(n), _) => s"Three entries; second is $n"
  case Seq(Some(1), _, _, _*) => "At least three entries, starting with 1"
  case Seq() => "Empty sequence"
  case _ => "Another sequence"
```

Patterns compose: list structure and Option cases can be matched in one decision.

Optional

Case blocks can be function arguments

```
val counts = List(("books", 3), ("pens", 5))

val labels = counts.map {
  case (name, n) => s"$name with $n items"
}
// List("books with 3 items", "pens with 5 items")

val increment: Option[Int] => Option[Int] = {
  case Some(n) => Some(n + 1)
  case None => None
}

increment(None)    // None
increment(Some(1)) // Some(2)
```

The expected type determines the function. map needs a result for every input.

Group payments, then match each map entry

```
val payments = List(
  Payment(BigDecimal(100), "CZK"),
  Payment(BigDecimal(40), "CZK"),
  Payment(BigDecimal(10), "EUR"))

val totals: Map[String, BigDecimal] =
  payments.groupBy(x => x.currency).map {
    case (currency, group) => currency -> group.map(x => x.amount).sum
  }
// Map("CZK" -> 140, "EUR" -> 10)
```

groupBy returns a Map of lists. The case block names the key and the group of payments.

Filtering entries and handling a missing key

```
val largeTotals = totals.collect {
  case (currency, amount) if amount >= 100 => currency -> amount
}
// Map("CZK" -> 140)

val message = totals.get("EUR") match
  case Some(amount) => s"EUR total: $amount"
  case None => "No EUR payments"
// "EUR total: 10"
```

collect selects matching entries. get returns an Option, which needs its own decision.

Patterns in for comprehensions

```
val rows: List[(String, Option[Int])] =
  List(("book", Some(3)), ("pen", None), ("map", Some(0)))

val labels =
  for {
    case (name, Some(n)) <- rows if n > 0
  } yield s"$name: $n"
// List("book: 3")

val byCount =
  for (name, n) <- Map("book" -> 3) yield n -> name
// Map(3 -> "book")
```

for case filters nonmatching rows. An irrefutable tuple pattern needs no case.

Case study: a simple calculator

```
// Expression forms
42              // number
x               // variable
x + y           // addition
x * y           // multiplication

// Our running example
(x * y) + (2 * 3)
// x = 10, y = 4
```

Build a language, then use it

Expr
Represent expressions as data.

eval(expr, env)
Evaluate to a number: 46.

fold(expr)
Simplify to (x * y) + 6.

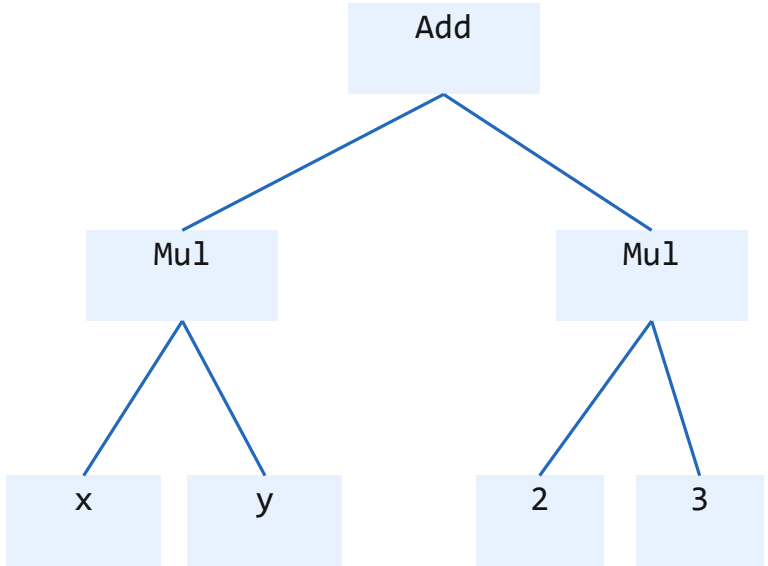
Define the syntax with an ADT; define its meaning with pattern matching.

Model the language with a recursive ADT

```
enum Expr:
  case Num(value: Double)
  case Var(name: String)
  case Add(left: Expr, right: Expr)
  case Mul(left: Expr, right: Expr)

import Expr.*

val expr =
  Add(Mul(Var("x"), Var("y")),
      Mul(Num(2), Num(3)))
```



Add and Mul contain Expr children. Leaves are numbers or variable names.

Bind a subtree while matching its shape

```
expr match
  case Add(left @ Mul(_, _), right) =>
    s"Left product: $left; right subtree: $right"
  case _ => "Another expression"

// left is Mul(Var("x"), Var("y"))
// right is Mul(Num(2), Num(3))
```

name @ pattern keeps the matched value as well as testing its structure.

Interpret the expression with eval

```
def eval(e: Expr, env: Map[String, Double]): Double = e match
  case Num(n) => n
  case Var(name) => env(name)
  case Add(l, r) => eval(l, env) + eval(r, env)
  case Mul(l, r) => eval(l, env) * eval(r, env)

val env = Map("x" -> 10.0, "y" -> 4.0)
eval(expr, env)   // 46.0
```

Each case gives one variant its meaning. The environment must contain every used variable.

Simplify by folding constants

```
def fold(e: Expr): Expr = e match
  case Add(l, r) =>
    (fold(l), fold(r)) match
      case (Num(a), Num(b)) => Num(a + b)
      case (a, b) => Add(a, b)
  case Mul(l, r) =>
    (fold(l), fold(r)) match
      case (Num(a), Num(b)) => Num(a * b)
      case (a, b) => Mul(a, b)
  case atom => atom
```

Fold children first. Collapse an operation when both resulting children are numbers.

Fold constants without changing the result

```
// Before
expr
// Add(
//   Mul(Var("x"), Var("y")),
//   Mul(Num(2), Num(3)))

eval(expr, env)   // 46.0
```

```
// After
val simplified = fold(expr)
// Add(
//   Mul(Var("x"), Var("y")),
//   Num(6))

eval(simplified, env)   // 46.0
```

The variables stay in the tree; the constant subtree 2 * 3 becomes Num(6).

ADTs and matching are used across language families

Learn the modelling idea once, recognize its encodings elsewhere.

Language	Product / alternatives	How matching differs
Haskell / OCaml	records / algebraic variants	Constructor patterns in functions
Rust	struct / enum with payloads	Exhaustiveness plus ownership
Java 21	records / sealed families	Record patterns and switch
C++17	struct / std::variant	std::visit in the standard library

Products, alternatives and decomposition recur; the syntax and guarantees differ.

Java before records: ordinary classes

```
abstract class Delivery {
  private Delivery() {}
  static final class Pickup extends Delivery {}
  static final class Courier extends Delivery {
    private final String address;
    private final boolean express;
    Courier(String address, boolean express) {
      this.address = address;
      this.express = express;
    }
    String address() { return address; }
    boolean express() { return express; }
  }
}
```

Java 8: a private base constructor and final nested cases close the family.

Java: records, sealed types and patterns

```
sealed interface Delivery permits Pickup, Courier {}

record Pickup() implements Delivery {}

record Courier(String address, boolean express) implements Delivery {}
class Fees {
  static int fee(Delivery d) {
    return switch (d) {
      case Pickup() -> 0;
      case Courier(var address, var express) when express -> 10;
      case Courier(var address, var express) -> 5;
      case null -> throw new IllegalArgumentException("Missing delivery");
    };
  }
}
```

Records supply products; sealed closes the sum. Record patterns and this switch use Java 21.

Haskell: constructors and lazy patterns

```haskell
data Delivery = Pickup
              | Courier String Bool

fee :: Delivery -> Int
fee Pickup = 0
fee (Courier _ True) = 10
fee (Courier _ False) = 5

courierToPrague = Courier "Prague"  -- Bool -> Delivery

ignore ~(Courier _ _) = 0
ignore undefined   -- 0: the lazy pattern is not inspected
fee undefined      -- runtime error when the value is demanded
```

Patterns appear in equations; constructors are functions. ~ delays a match in a lazy language.

Optional

OCaml: match values, control construction

```
(* ordinary variant type *)
type delivery =
  | Pickup
  | Courier of string * bool

let fee = function
  | Pickup -> 0
  | Courier (_, true) -> 10
  | Courier (_, false) -> 5

fee (Courier ("Prague", true))
(* 10 *)
```

```
(* delivery.mli: module interface *)
type t = private
  | Pickup
  | Courier of string * bool
val pickup : t
val courier : string -> bool -> t

(* client code *)
let is_pickup = function
  | Delivery.Pickup -> true
  | Delivery.Courier (_, _) -> false

let bad = Delivery.Pickup (* error *)
```

A private variant allows client pattern matching but requires construction through the module API.

Optional

Rust: exhaustive matching can borrow

```
enum Delivery {
    Pickup,
    Courier { address: String, express: bool },
}

fn fee(d: &Delivery) -> u32 {
    match d {
        Delivery::Pickup => 0,
        Delivery::Courier { express: true, .. } => 10,
        Delivery::Courier { express: false, .. } => 5,
    }
}
```


Optional

C++: ADTs today and proposed pattern matching

```
#include <string>
#include <variant>
struct Pickup {};
struct Courier { std::string address; bool express; };
using Delivery = std::variant<Pickup, Courier>;

struct Fee {
  int operator()(const Pickup&) const { return 0; }
  int operator()(const Courier& c) const { return c.express ? 10 : 5; }
};
int fee(const Delivery& d) {
  return std::visit(Fee{}, d);
}
```

struct + variant already encode ADTs. Native matching missed C++26; it is a C++29 priority.

WG21 P5000R1 (2026), §4.1: pattern matching is a C++29 priority

Optional

Pattern matching: make the failure behavior deliberate

Inspect and decide

Patterns test shape and bind names.
Cases run from top to bottom.
Guards can leave cases uncovered.

Handle each meaningful ADT case.

Use exhaustive decisions for domain cases; use filtering when discarding inputs is intended.

Summary: put a boundary around the rule

```
val seats = SeatPool(3)
seats.tryReserve(2)   // true
seats.tryReserve(2)   // false
seats.available       // 1
```

State an invariant.

Expose operations that preserve it.

Control aliases to mutable state.

Hide storage choices from callers.

Encapsulation makes the contract stable and reasoning about its implementation local.

Summary: choose the values and their cases

```
final case class Payment(
  amount: BigDecimal,
  currency: String)

enum Delivery:
  case Pickup
  case Courier(
    address: Address,
    express: Boolean)
```

Group facts in products.

Represent alternatives with sums.

Validate fields at the boundary.

Use patterns to inspect and transform the resulting values.

Objects protect operations; case classes and ADTs expose useful information. They work together.

Questions

- What is encapsulation?
- Why is it useful?
- How to model plain data?
- When to use plain data classes?

Opaque types

With Long, time units are easy to mix up

```
def sleep(delay: Long): Unit =
  Thread.sleep(delay)

val delay: Long = 200L + 300L

delay / 1000.0   // 0.5 seconds
sleep(delay)     // 500 milliseconds
```

```
val seconds: Long = 5L
sleep(seconds)   // 5 ms, not 5 s

val timestamp = System.currentTimeMillis()
sleep(timestamp) // timestamp as duration

sleep(-1L)       // compiles; throws
```

Milliseconds, seconds and timestamps are all Long. The compiler cannot tell them apart.

Give the opaque duration its own operations

```
object Time:
  opaque type Millis = Long
  object Millis:
    def apply(n: Long): Millis =
      require(n >= 0)
      n
  extension (m: Millis)
    def toLong: Long = m
    def toSeconds: Double = m / 1000.0
    def +(other: Millis): Millis =
      Millis(Math.addExact(m, other))
```

```
import Time.*

def sleep(delay: Millis): Unit =
  Thread.sleep(delay.toLong)

val delay = Millis(200) + Millis(300)

delay.toSeconds   // 0.5
sleep(delay)

sleep(5L)         // compile error
Millis(-1L)       // rejected
```

Extensions preserve the unit. Convert to Long at the Thread.sleep boundary.